Helmet Compliance Monitor: Technical Report

Lokesh Singh, CV Raman Global University, Bhubaneswar. OpenCV AI Competition 2026, powered by AWS.

Code repository: https://github.com/lokeshhhsingh/helmet-compliance-monitor

Live demo: http://43.205.33.42 (kept running through the judging period, until about November 10, 2026). Demo video: https://youtu.be/mEgFhqNYVmM

1. Summary

Helmet Compliance Monitor takes traffic footage and finds two-wheeler riders who are not wearing a helmet. It tracks every bike, judges **each head on the bike separately** (so a bare-headed passenger behind a helmeted driver can be caught), and confirms a violation only after a second, enlarged look agrees. Cases it cannot judge, such as very dark or unclear heads, are sent to human review instead of being accused. For each confirmed violation it saves an evidence photo to Amazon S3, writes a record to DynamoDB and sends an email alert through Amazon SNS.

On 8 short stock clips, the system flagged 4 of the 5 people who were clearly bare-headed, missed 1, did not detect a child who appeared to have no helmet, and raised 1 flag that we could not confirm. It runs at about 2.8 times real time on a CPU-only server. This report states these failures plainly, because the system is meant to support a human reviewer and not to replace one.

2. Problem and intended users

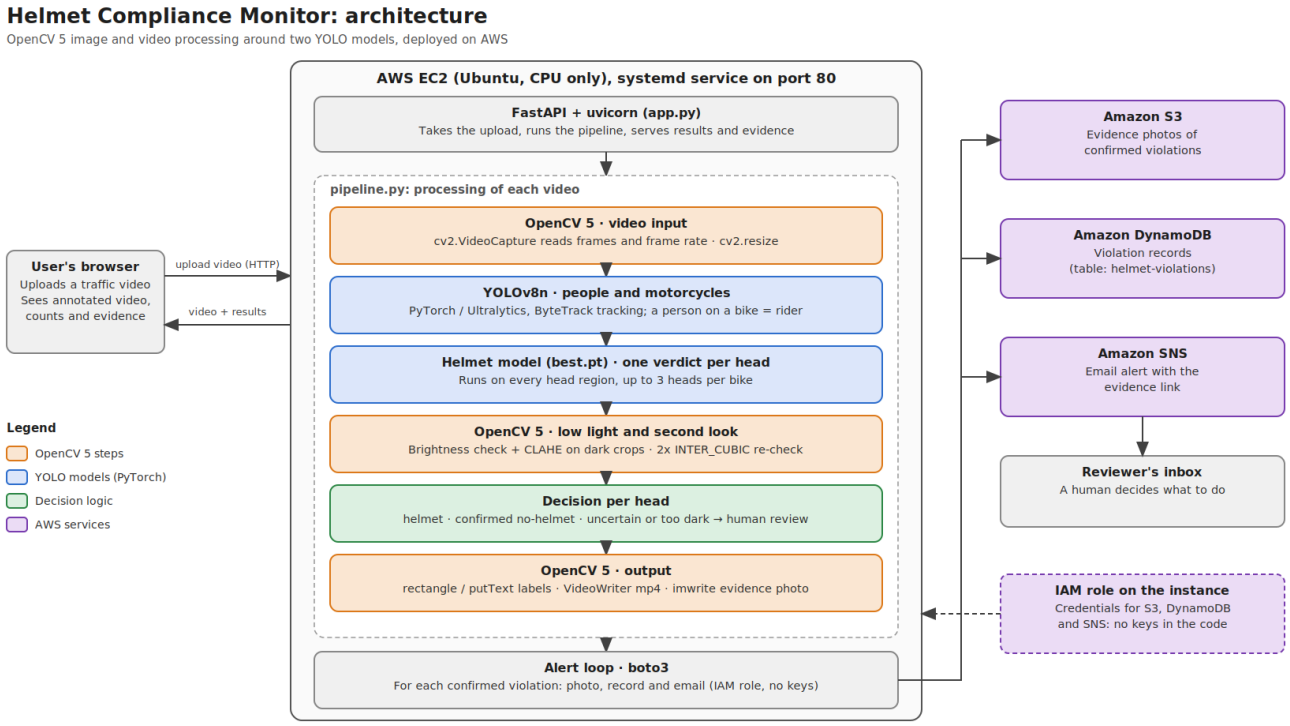
Riding without a helmet greatly increases the risk of serious head injury for two-wheeler riders. Checking footage by hand is slow, and the hard cases are the ones a simple detector gets wrong: two or three people on one bike, a child seated in front, riders partly hidden behind others, and poor light.

Intended users. Road-safety teams, traffic authorities and city officials who already collect roadside video, and safety officers at campuses, factories and construction sites with fixed cameras. The system is a triage tool. It narrows long footage to a short list of moments, each with an evidence photo and a plain-language reason, so that a person can check them.

3. How the system works

For every video the pipeline does the following.

1. **Find riders.** A COCO-pretrained YOLOv8n model detects people and motorcycles, and ByteTrack gives each one a stable ID across frames. A rider is a tracked person overlapping a tracked motorcycle. The code also re-links a new tracker ID that appears close to where a bike just vanished, so a short tracking break does not create a second bike.
2. **Judge each head.** A helmet model that we trained (YOLOv8n, two classes: `helmet` and `no-helmet`) runs on the upper part of each bike. Up to 3 heads per bike are tracked (driver plus passengers) and labelled `1A`, `1B`, and so on. A head must be detected 8 times before it counts as a person, which blocks one-off phantom detections.
3. **Candidate for a violation.** A head becomes a candidate when at least 70% of its detection confidence says `no-helmet`, it has at least 8 no-helmet detections, and it has been tracked for about one second.
4. **Second look.** Stored crops of the head are enlarged 2x and judged again about 0.2 seconds apart. The violation is confirmed only if at least 3 crops were usable and at least 60% of them agree. The evidence photo records the result, for example "5/5 re-checks agreed".
5. **Review path.** Heads that lean towards no-helmet but are not confirmed are marked `uncertain` and go to a review list. Each review case is saved to DynamoDB with the status needs_review and a reason ("not enough clear views" or "low light"). It does not trigger an email, so only confirmed violations send alerts. Heads in very dark crops are never confirmed automatically and go to review with the reason "low light".
6. **Act.** For each confirmed violation, the alert loop stores the evidence photo in S3, writes the record to DynamoDB and sends the email through SNS. The visual result decides what the system does next.



4. Where OpenCV 5 is used

OpenCV 5.0.0.93 does the video and image processing around the models. The detection models themselves run on PyTorch through Ultralytics, not through OpenCV's DNN module.

- **Video input and output.** `cv2.VideoCapture` reads the frames and the clip's frame rate, size and frame count. `cv2.VideoWriter` writes the annotated mp4.
- **Image preparation.** Frames are resized with `cv2.resize`. For the second look, head crops are enlarged 2x with bicubic interpolation (`cv2.INTER_CUBIC`).
- **Low-light handling.** A crop's brightness is the mean of its grayscale values (0 to 255). Crops darker than 90 are converted to LAB colour with `cv2.cvtColor`, their lightness channel is contrast-enhanced with CLAHE (`cv2.createCLAHE`, clip limit 3.0, 4x4 tiles) and they are converted back. This happens both when a head is first judged and in the second look. Heads darker than 45 are too dark to judge and are never confirmed automatically.
- **Evidence.** `cv2.rectangle` and `cv2.putText` draw the bike and head boxes with their labels. `cv2.hconcat` joins the re-check crops into one strip, and `cv2.imwrite` saves the evidence photo.

5. Models and training

The system uses two models and only one of them was trained by us.

- **Scene model (not trained by us).** The COCO-pretrained `yolov8n.pt` from Ultralytics, used to find people and motorcycles.
- **Helmet model (trained by us).** A YOLOv8n model fine-tuned from the COCO-pretrained weights (319 of 355 weight tensors were transferred, and the detection head was re-initialised for 2 classes). It has about 3.0 million parameters and 8.1 GFLOPs, and the saved `best.pt` is 6.3 MB.

Dataset. The public Roboflow Universe project "helmet_detection" by the user sala, licensed CC BY 4.0 (the exact page address was not recorded; the project can be found by name on Roboflow Universe). It contains 19,212 images in the version we used (the Universe page says 19,214), with two classes, `helmet` and `no-helmet`. It was forked into our own workspace and exported in YOLOv8 format with auto-orientation and a resize (stretch) to 640x640 pixels. We added no augmentation beyond the Ultralytics defaults. The split was 15,212 training, 2,709 validation and 1,291 test images. A small number of images with mixed polygon and box labels were skipped by the trainer.

Training. 50 epochs at image size 640 with batch size 16 and the Ultralytics default settings and augmentations, on a free Colab T4 GPU (Ultralytics 8.4.171), taking 3.2 hours. `best.pt` was chosen using the validation scores.

Held-out scores (test split: 2,138 heads, of which 1,341 are helmets and 797 are no-helmet):

Class	Precision	Recall	mAP50	mAP50-95
All classes	0.877	0.890	0.917	0.616
helmet	0.912	0.931	0.960	0.692
no-helmet	0.841	0.849	0.875	0.540

Validation mAP50 for all classes was 0.919, and we headline the test number. Inference took about 2 to 4 ms per image on the T4.

Two cautions about these numbers. First, the dataset page does not say where the original images came from, and many file names look like frames cut from videos. If frames from one video fall into both the training and test splits, these scores overstate how the model performs on new footage. Second, the weaker class is `no-helmet`, the one that matters most: its precision of 0.841 means that about 1 in 6 of its detections was wrong on this test set. This is why the pipeline never relies on a single detection and adds the second look and the review path. The 8-clip evaluation in Section 7, which uses separate stock footage, is the more realistic evidence.

6. AWS architecture and deployment

- **Amazon EC2** (Ubuntu, CPU only, region ap-south-1 Mumbai) runs the web app and the pipeline. The app is FastAPI served by uvicorn on port 80, started as a systemd service that restarts automatically.
- **Amazon S3** stores the evidence photo of each confirmed violation.
- **Amazon DynamoDB** (table `helmet-violations`) stores one record per confirmed violation, and one for each review case (status needs_review).
- **Amazon SNS** sends the email alert, which contains the second-look result and a link to the evidence photo.
- **AWS IAM role** attached to the instance gives the code access to S3, DynamoDB and SNS. No access keys are stored in the code or the repository.
- **Settings** (bucket, topic, table, region and public address) are environment variables kept in a systemd override file on the server, not in the repository.

Alerts are sent when a clip finishes processing, not at the instant a violation is detected. In our tests both alerts for a two-violation clip arrived in the same minute.

Deployment steps and testing are in the README: pinned versions in `requirements.txt` (CPU-only PyTorch), setup, running one clip, running the web app, and the expected output on a reference clip.

7. Evaluation

We ran the full system on 8 short clips of free stock footage and wrote down the true answer for each by hand. Eight clips can show how the system behaves. They cannot measure its accuracy.

Clip	Truth	System	Result
Bare-headed pair (driver and passenger)	2 riders, both without helmet	2 riders, 2 violations	correct
Night rider	1 rider, helmet	1 rider, no violation	correct
Solo rider (62 s)	1 rider, no helmet, alone throughout	1 rider, 1 violation	correct
Bearded man, front view	1 rider, no helmet	1 rider, 1 violation	correct

Clip	Truth	System	Result
Racing bikes	3 riders, all helmets	7 riders, 0 violations	verdicts right, rider count too high
Scooter group	about 18 people, helmets (one rider unclear)	18 riders, 1 violation, 2 sent to review	count about right, 1 flag unverified
Close-up, parked bike	1 rider, no helmet	0 riders	**miss**
Scooter with a child in front	rider with helmet, child apparently without	rider found, child not found	**child missed**

Summary. 5 people were clearly bare-headed. The system flagged 4 of them and missed 1 (the close-up). It also did not detect a child who appeared to have no helmet, and it raised 1 flag we could not confirm. The unclear rider in the scooter group was correctly sent to human review and not accused.

Speed. On the CPU server, a 20.4 s clip at 720p took 57.5 s and a 62 s clip took 2 min 51 s, about 2.8 times real time (roughly 8.7 frames per second). As a baseline, the earlier version of the pipeline, which judged one head per bike, took about 53 s on the same 20.4 s clip. Judging every head costs about 8% more time and is what lets the system catch the bare-headed passenger.

Test footage. All clips are free videos from Pexels, used under the Pexels License.

Clip	Creator	Link
Bare-headed pair	cottonbro studio	https://www.pexels.com/video/a-couple-riding-a-motorbike-5207050/
Night rider	Mario Spencer	https://www.pexels.com/video/nighttime-motorcycle-ride-under-city-overpass-31909050/
Close-up, parked bike	Kindel Media	https://www.pexels.com/video/a-man-riding-a-motorbike-8551058/
Bearded man, front view	cottonbro studio	https://www.pexels.com/video/a-man-riding-a-big-bike-5198984/
Scooter group	setengah lima sore	https://www.pexels.com/video/group-ride-of-classic-scooters-on-scenic-road-32189923/

Clip	Creator	Link
Scooter with a child in front (vertical video)	setengah lima sore	https://www.pexels.com/video/father-and-son-scooter-ride-in-indonesia-33635200/
Solo rider (62 s)	K	https://www.pexels.com/video/a-man-riding-a-motorcycle-in-the-neighborhood-4310618/
Racing bikes	Harvey Tan Villarino	https://www.pexels.com/@harveyvillarino/ (creator page; exact clip page to be confirmed)

8. Limitations

- **Close-ups.** The system expects the whole motorcycle to stay visible. In a close-up of a parked bike, the motorcycle was detected only in short flashes, never for the one second needed to form a rider, so a bare-headed person was not flagged.
- **Hidden passengers.** A small child seated in front of the rider, partly hidden by the rider's arm, was not detected. The helmet model rarely finds heads that small or occluded, so a possible missing helmet went unflagged.
- **Fast racing footage.** On riders leaning through corners, the system reported 7 riders for 3 real ones. All were judged as wearing helmets, so there was no false alarm, but rider counts are unreliable when tracking breaks, and a duplicate rider judged as having no helmet could in principle produce a false violation.
- **Dense groups and small heads.** In a group of about 18 scooter riders, one small, partly hidden head was confirmed as "no helmet". It was reported under one bike but may belong to a neighbouring bike, and we could not tell by eye whether that rider wears a helmet, so the flag is unverified. The second look enlarges the same small image, so it adds little for tiny heads.
- **Speed.** About 2.8 times real time on a CPU instance, so live camera streams would need a GPU or a faster instance. Alerts are sent after a clip finishes.
- **Training data.** The origin of the training images is undocumented, and frames from the same video may appear in both the training and test splits.
- **Small evaluation.** Eight clips, labelled by one person.

9. Responsible use and privacy

- **A human decides.** The system flags footage for review. It does not issue penalties, and all wording avoids terms like "offender".

- **Avoiding false accusations.** A violation needs a candidate head, then an enlarged second look that must agree. Unclear and dark cases go to review. Even so, a head-level no-helmet precision of 0.841 and one unverified flag in our evaluation show that false flags can happen, which is why a person must check each one.
- **Faces and evidence.** Evidence photos can show identifiable faces. We used only licensed stock footage of people who are not real offenders. Evidence is deleted automatically 21 days after it is created: an S3 lifecycle rule removes the stored photos, and a nightly job on the server removes the uploaded videos and the copies that the app serves through the email links. Evidence links are still served over plain HTTP and are not individually protected, so anyone who has a link can open that photo until it is deleted, and the demo page has no login, so anyone who knows its address can upload a video. It is meant for the judging period only. Before any real-world use, the system would need HTTPS, access control or expiring links, and face blurring.
- **Access.** AWS access uses an IAM role, and no credentials are stored in the code or the repository.
- **Rights.** The test clips come from Pexels under its license. The helmet dataset is CC BY 4.0 and is credited above, but its original image sources are not documented, so we cannot confirm the consent or rights of every person pictured in it.
- **No overclaiming.** The report states every failure we found, and the README repeats the limitations.

10. Future work

1. Retrain the helmet model with examples of children, partly hidden heads and small distant heads.
2. Send heads below a minimum size to human review instead of confirming them automatically.
3. Merge duplicate riders when tracking breaks, to fix the over-count on fast footage.
4. Add HTTPS, a login for uploads, expiring or access-controlled evidence links, and face blurring.
5. Support live camera streams by moving to a GPU or a faster instance.
6. Evaluate on a larger set of footage, labelled independently, with a clean split between training and test videos.

11. Reproducing the results

The repository contains the code, the trained `best.pt`, pinned dependencies (`requirements.txt`), the architecture diagram, and build, deployment and test instructions. The saved reference output for the bare-headed pair clip is

`results/bare_head_perhead.json`: 1 bike, 2 riders, 2 violations.

12. Credits

- Helmet dataset: "helmet_detection" by sala on Roboflow Universe, CC BY 4.0 (page address not recorded; search Roboflow Universe by project name).
- Models and libraries: Ultralytics YOLOv8, PyTorch, OpenCV 5.0.0.93, FastAPI, uvicorn, boto3.
- Test footage: Pexels creators listed in Section 7.